

The monoref package*

Srikanth Mohankumar
srikanthmohankumar@gmail.com

September 28, 2026

Abstract

`monoref` resolves forward cross-references, page references, plain numbered citations, the total page count and a multi-level table of contents in a *single* Lua $\text{\LaTeX}$ run. No second pass, no `latexmk`, and no `.aux` round-trip are required. It holds every finished page in memory and back-patches the still unknown values at `\end{document}`. When `hyperref` is loaded, the references and contents entries become clickable.

1 Introduction

Normally $\text{\LaTeX}$ needs at least two runs to settle cross-references: the first writes label values, section numbers and page numbers to the `.aux` file and the second reads them back. This package removes that round-trip for a useful subset of the problem by never shipping a page out immediately. Every page is *held* in memory (using the `shipout/before` hook and `\DiscardShipoutBox`); at the end of the document all values are known, so they can be patched into the held pages before the pages are finally shipped.

Two ideas make it work.

Reserved slots. A value that is not yet known (a forward `\ref`, any `\pageref`, `\lastpage`) is typeset as a reserved box. At flush the box collapses to the value's natural width and the enclosing line is re-justified to its original width, so the interword glue absorbs any freed space: short values (a single digit) leave no gap. The reserved width therefore only guides the original line break, not the final spacing.

Shipout-accurate pages. `\label` drops an invisible `\special` marker; when a page is held it is scanned for markers and each label is stamped with the page it really landed on. This avoids the off-by-one you get from reading `\value{page}` at `\label` time, because $\text{\TeX}$'s page builder is asynchronous.

2 Requirements

`monoref` requires Lua $\text{\LaTeX}$. It stops with an error on any other engine.

*This document corresponds to `monoref` v1.4-dev, dated September 28, 2026.

3 Usage

```
\documentclass{article}
\usepackage{hyperref}      % optional; enables clickable links
\usepackage{monoref} % either load order works (v1.4)
\begin{document}
\section{Introduction}\label{sec:intro}
See Section~\ref{sec:results} on page~\pageref{sec:results}.
This document has \lastpage\ body pages.
...
\section{Results}\label{sec:results}
...
\end{document}
```

- `\ref` Prints the number of the referenced `\label`. A backward reference is typeset immediately; a forward reference is emitted as a slot and filled at the end.
- `\pageref` Prints the page of the referenced `\label`. It is always a slot, because a label's true page is known only once its page is assembled.
- `\lastpage` Prints the total number of body pages (front matter excluded).
- `\cite` Prints the citation label(s) of one or more `\bibitem` keys, in source order, formatted through the class's `\@cite` (brackets in the standard classes). A citation before the bibliography is a forward reference and resolves in the same single run. See the Citations section below.

4 Citations

From v1.4 the package resolves plain- $\text{\LaTeX}$ numbered citations in the same single run. Every `\bibitem` registers its citation label (the number, or the optional `[label]` text) under the hood, and `\cite` goes through the same slot machinery as `\ref`: a citation *after* its `\bibitem` is typeset immediately, a citation *before* it – the normal case, with the bibliography at the end of the document – becomes a reserved slot that is patched at flush. Multiple keys (`\cite{a,b}`) are printed in source order with the kernel's separator, exactly as plain $\text{\LaTeX}$ does; the optional note (`\cite[p.~5]{a}`) is supported. With `hyperref` loaded, citations link to their bibliography entries.

This covers a hand-written `thebibliography`. A $\text{\BibTeX}$ -generated `.bbl` also works once it exists (its `\bibitems` register normally when `\bibliography` reads it), but generating or refreshing the `.bbl` still requires running $\text{\BibTeX}$, which is outside the single-run scope.

What is *not* provided: sorting or range compression of citation lists, author-year styles, and the `natbib`, `cite` and `biblatex` interfaces. If one of those packages is loaded, `monoref` leaves `\cite` alone (their `.aux`-based behaviour, needing reruns, is unchanged); the package option `nobib` disables the citation support explicitly.

5 Slot width

Forward values are reserved with a font-relative width taken from a template string. This width only guides the original line break; at flush the slot collapses to the value's natural width and the line is re-justified, so the reserved

`\monorefreftemplate` width never shows up as a gap. `\monorefreftemplate` (default 0.0.0) sizes `\monorefpagetemplate` `\ref` slots; it must be at least as wide as the widest forward (sub)section number. `\monorefpagetemplate` (default 000) sizes `\pageref` and `\lastpage` slots. `\monorefcitetemplate` `\monorefcitetemplate` (default 00) sizes forward `\cite` slots; it must be at least as wide as the widest citation label. Redefine any of them with `\renewcommand`, e.g. `\renewcommand\monorefreftemplate{0.0}` for a document with no forward `\monorefslotwidth` subsubsection references. Setting the length `\monorefslotwidth` to a positive value overrides both templates with a fixed width.

A replacement *narrower* than its reserved slot is harmless by design: the slot collapses and the line's glue absorbs the slack. A replacement *wider* than the slot cannot be absorbed, so since v1.4 the flush phase emits a warning naming the offending command, the page, the overshoot in points, and the template to widen.

6 Hyperlinks

If `hyperref` is loaded, `\ref`, `\pageref` and the contents entries become clickable. The links use the package's own named destinations (derived directly from the label names), *not* `hyperref`'s `.aux`-based anchors, so linking stays single-pass and works for forward references too. Since v1.4 the load order of `monoref` and `hyperref` does not matter: the package takes over the reference commands in the `begindocument/end` hook, after every package's begin-document code has run.

7 The table of contents

Without the `notoc` option a contents list covering `\section`, `\subsection` and `\subsubsection` is collected during the run and prepended at the end with roman folios, while the body keeps arabic numbering; prepending the contents therefore does not renumber the body and there is no circular dependency. Use `notoc` to switch it off.

8 Limitations

- It holds the whole document in memory until `\end{document}`.
- It redefines `\label`, `\ref`, `\pageref`, `\section`, `\subsection` and `\subsubsection`; because it replaces `hyperref`'s versions of these, `hyperref` features that depend on them (`\autoref`, `\nameref`, `cleveref`) are not available. It also conflicts with `titlesec` and `varioref`.
- A forward value wider than its slot template overflows the line (a warning names the slot and the template to widen).
- Numbered-equation `\ref` works, including with `amsmath`; but `\eqref` (the parenthesised form) and lists of figures/tables are not provided.
- `\cite` covers plain- $\text{\LaTeX}$ numbered citations only: no sorting or range compression, no author-year styles, and no `natbib/cite/biblatex` (if one of those is loaded, `\cite` is left to it and needs reruns as usual).

- Because it intercepts every `\shipout`, it is incompatible with other packages that manipulate shipout.

9 Examples

The distribution ships a set of runnable examples in the `examples/` directory; compile each once with Lua^AT_EX:

01-minimal the smallest possible use.

02-article three heading levels, a numbered equation, forward and backward `\ref/\pageref`, a page total, and a front TOC.

03-hyperref-links clickable links via `hyperref` (loaded first, with `hidelinks`).

04-notoc the `notoc` option.

05-page-of-total a “Page X of Y” footer with `fancyhdr` and `\lastpage`.

06-custom-slot-width widening the slot templates.

07-citations single-run numbered `\cite` with a hand-written `thebibliography`: forward, repeated and multi-key citations.

10 Implementation: the style file

```

1 <*package>
2 \NeedsTeXFormat{LaTeX2e}
3 \ProvidesPackage{monoref}%
4   [2026/07/31 v1.4-dev Single-pass cross-references, page totals, TOC and citations (LuaLaTeX)]
5
6 \RequirePackage{iftex}
7 \RequireLuaTeX
8 \RequirePackage{xparse}
9
10 \newif\ifmonoref@dotoc \monoref@dotoctrue
11 \DeclareOption{notoc}{\monoref@dotocfalse}
12 \newif\ifmonoref@dobib \monoref@dobibtrue
13 \DeclareOption{nobib}{\monoref@dobibfalse}
14 \ProcessOptions\relax
15
16 \directlua{if not pcall(require, "monoref") then
17               dofile("monoref.lua") end}
18
19 \newbox\monoref@shipbox
20 \directlua{monoref.shipreg = \number\monoref@shipbox}
21 \attributedef\monoref@slotattr=\directlua{tex.print(monoref.slotattr)}\relax
22 \attributedef\monoref@markattr=\directlua{tex.print(monoref.markattr)}\relax
23
24 \newcommand\monorefreftemplate{0.0.0}
25 \newcommand\monorefpagetable{000}
26 \newcommand\monorefcitetemplate{00}
27 \newlength\monorefslotwidth \setlength\monorefslotwidth{0pt}

```

```

28 \newlength\monoref@tmpwd
29
30 \newcommand\monoref@slot[2]{%   #1 = slot id   #2 = width template
31   \begingroup
32     \ifdim\monorefslotwidth>\z@ \monoref@tmpwd=\monorefslotwidth
33     \else \settowidth\monoref@tmpwd{#2}\fi
34     \monoref@slotattr=#1\relax \hbox to \monoref@tmpwd{\hfil}%
35   \endgroup}
36
37 \newcommand\monoref@mark[1]{\monoref@markattr=#1\relax \special{}}
38
39 \newif\ifmonoref@hyper \monoref@hyperfalse
40 \newcommand\monoref@target[1]{\ifmonoref@hyper\hypertarget{monoref:#1}\fi}
41 \newcommand\monoref@link[2]{\ifmonoref@hyper\hyperlink{monoref:#1}{#2}\else#2\fi}
42
43 \newcommand\lastpage{\monoref@slot{\directlua{monoref.newslot("", "total")}}{\monorefpagetempl
44
45 \newcommand\monoref@refinner[1]{%
46   \ifnum\directlua{monoref.known("#1")}>\z@
47     \directlua{monoref.ref_value("#1")}%
48   \else
49     \monoref@slot{\directlua{monoref.newslot("#1", "ref")}}{\monorefreftemplate}%
50   \fi}
51
52 \newcommand\monoref@biblabel[2]{%   #1 = cite key   #2 = printed label value
53   \protected@edef\monoref@cbl{#2}%
54   \monoref@mark{\directlua{monoref.label_begin("mbib:\luaescapestring{#1}",
55     "\luaescapestring{\monoref@cbl}")}}%
56   \monoref@target{lbl:mbib:#1}}
57
58 \newcommand\monoref@citeinner[1]{%
59   \ifnum\directlua{monoref.known("mbib:\luaescapestring{#1}")}>\z@
60     \directlua{monoref.ref_value("mbib:\luaescapestring{#1}")}%
61   \else
62     \monoref@slot{\directlua{monoref.newslot("mbib:\luaescapestring{#1}", "ref")}}%
63     {\monorefcitetemplate}%
64   \fi}
65
66 \newcommand\monoref@citelist[1]{%   keys in source order, kernel separator
67   \let\monoref@sep\@empty
68   \@for\monoref@cb:=#1\do{%
69     \edef\monoref@cb{\expandafter\@firstofone\monoref@cb\@empty}%
70     \if@files\immediate\write\@auxout{\string\citation{\monoref@cb}}\fi
71     \monoref@sep
72     \monoref@link{lbl:mbib:\monoref@cb}{\monoref@citeinner{\monoref@cb}}%
73     \def\monoref@sep{,\penalty\@m\ }%
74
75 \newcommand\monoref@record[3]{%   #1 level   #2 number   #3 title
76   \protected@edef\monoref@nm{#2}%
77   \protected@edef\monoref@tl{#3}%
78   \monoref@mark{\directlua{monoref.toc_begin(#1,
79     "\luaescapestring{\monoref@nm}", "\luaescapestring{\monoref@tl}")}}%
80   \ifmonoref@hyper
81     \edef\monoref@ti{\directlua{monoref.toc_count()}}%

```

```

82     \hypertarget{monoref:toc:monoref@ti}{}%
83     \fi}
84
85 \def\monoref@activate{%
86   \@ifpackageloaded{hyperref}{\monoref@hypertrue}{\monoref@hyperfalse}%
87   \let\monoref@origlabel\label
88   \let\monoref@origsection\section
89   \let\monoref@origsubsection\subsection
90   \let\monoref@origsubsubsection\subsubsection
91   \RenewDocumentCommand\label{m}{%
92     \protected@edef\monoref@c1{\@currentlabel}%
93     \monoref@mark{\directlua{monoref.label_begin("##1","\luaescapestring{\monoref@c1}")}}}%
94     \monoref@target{lbl:##1}%
95     \monoref@origlabel{##1}}%
96   \RenewDocumentCommand\ref{m}{\monoref@link{lbl:##1}{\monoref@refinner{##1}}}%
97   \RenewDocumentCommand\pageref{m}{%
98     \monoref@link{lbl:##1}{\monoref@slot{\directlua{monoref.newslot("##1","page")}}}{\monoref@refinner{##1}}}%
99   \RenewDocumentCommand\section{s o m}{%
100     \IfBooleanTF{##1}{\monoref@origsection*{##3}}%
101     {\IfValueTF{##2}{\monoref@origsection[##2]{##3}}{\monoref@origsection{##3}}}%
102     \monoref@record{1}{\thesection}{##3}}%
103   \RenewDocumentCommand\subsection{s o m}{%
104     \IfBooleanTF{##1}{\monoref@origsubsection*{##3}}%
105     {\IfValueTF{##2}{\monoref@origsubsection[##2]{##3}}{\monoref@origsubsection{##3}}}%
106     \monoref@record{2}{\thesubsection}{##3}}%
107   \RenewDocumentCommand\subsubsection{s o m}{%
108     \IfBooleanTF{##1}{\monoref@origsubsubsection*{##3}}%
109     {\IfValueTF{##2}{\monoref@origsubsubsection[##2]{##3}}{\monoref@origsubsubsection{##3}}}%
110     \monoref@record{3}{\thesubsubsection}{##3}}%
111   \@ifpackageloaded{amsmath}{\let\ltx@label\label}{}%
112   \ifmonoref@dobib
113     \@ifpackageloaded{natbib}{\monoref@dobibfalse}{}%
114     \@ifpackageloaded{cite}{\monoref@dobibfalse}{}%
115     \@ifpackageloaded{biblatex}{\monoref@dobibfalse}{}%
116   \fi
117   \ifmonoref@dobib
118     \let\monoref@origbibitem\bibitem
119     \def\bibitem{\@ifnextchar[\monoref@lbibitem\monoref@bibitem]}
120     \def\monoref@lbibitem[##1]##2{%
121       \monoref@origbibitem[##1]{##2}\monoref@biblabel{##2}{##1}}%
122     \def\monoref@bibitem##1{%
123       \monoref@origbibitem{##1}\monoref@biblabel{##1}{\@currentlabel}}%
124     \RenewDocumentCommand\cite{o m}{%
125       \IfValueTF{##1}{\@tempwattrue}{\@tempwafalse}%
126       \@cite{\monoref@citelist{##2}}{\IfValueTF{##1}{##1}{}}}%
127   \else
128     \PackageInfo{monoref}{Citation support off
129       (nobib option, or natbib/cite/biblatex detected)}%
130   \fi
131 }
132 \AddToHook{begindocument/end}{\monoref}{\monoref@activate}
133
134 \newcommand\monoref@indent[1]{\ifcase#1\or\or\hspace{1.5em}\or\hspace{3em}\fi}
135 \newcommand\monoref@tocline[5]{% #1 idx #2 level #3 number #4 title #5 page

```

```

136 \par\noindent\monoref@indent{#2}%
137 \monoref@link{toc:#1}{\makebox[2.6em][l]{#3}#4}\dotfill\ #5}
138
139 \newcount\monoref@nt \newcount\monoref@k
140 \newcommand\monoref@buildtoc{%
141   \directlua{monoref.target = "toc"}%
142   \pagenumbering{roman}%
143   \monoref@nt=\directlua{monoref.toc_count()}\relax
144   \ifnum\monoref@nt>\z@
145     \section*{\contentsname}\par\medskip
146     \monoref@k=\z@
147     \loop\ifnum\monoref@k<\monoref@nt
148       \advance\monoref@k\@ne
149       \monoref@tocline
150       {\the\monoref@k}%
151       {\directlua{monoref.toc_level(\the\monoref@k)}}%
152       {\directlua{monoref.toc_num(\the\monoref@k)}}%
153       {\directlua{monoref.toc_title(\the\monoref@k)}}%
154       {\directlua{monoref.toc_page(\the\monoref@k)}}%
155     \repeat
156     \clearpage
157   \fi}
158
159 \newif\ifmonoref@flushing
160 \AddToHook{shipout/before}{%
161   \ifmonoref@flushing\else
162     \directlua{monoref.hold(\number\ShipoutBox, \the\value{page})}%
163     \DiscardShipoutBox
164   \fi}
165
166 \newcount\monoref@i
167 \AtEndDocument{%
168   \clearpage
169   \ifmonoref@dotoc \monoref@buildtoc \fi
170   \directlua{monoref.finalize()}%
171   \monoref@i=\directlua{monoref.qcount()}\relax
172   \monoref@flushingtrue
173   \loop\ifnum\monoref@i>\z@
174     \directlua{monoref.qpop()}%
175     \shipout\box\monoref@shipbox
176     \advance\monoref@i\@m@ne
177   \repeat
178   \monoref@flushingfalse}
179
180 \end{package}

```

11 Implementation: the Lua backend

The backend is a plain Lua module, generated verbatim into `monoref.lua`. Its own comment header carries the licence (a `docstrip %%` header would be invalid Lua), so it is shipped standalone rather than generated by the installer.

```

181 (*lua)
182 -- monoref.lua
183 -- Lua backend for the monoref package.

```

```

184 --
185 -- Copyright (C) 2026 Srikanth Mohankumar <srikanthmohankumar@gmail.com>
186 -- This work may be distributed and/or modified under the conditions of
187 -- the LaTeX Project Public License, version 1.3c or later.
188 -- http://www.latex-project.org/lppl.txt
189
190 monoref = monoref or {}
191
192 monoref.labels = {} -- name -> {ref=, page=}
193 monoref.slots = {} -- id -> {name=, font=, kind=} kind: ref | page | total
194 monoref.marks = {} -- id -> {kind=, key=} kind: label | toc
195 monoref.toc = {} -- ordered list of {level=, num=, title=, page=}
196 monoref.held = {} -- body pages (arabic)
197 monoref.toc_pages = {} -- front-matter pages (roman)
198 monoref.queue = {} -- final ship order
199 monoref.target = "body"
200 monoref.sid, monoref.mid, monoref.total = 0, 0, 0
201
202 monoref.slotattr = luatexbase.new_attribute("monorefslot")
203 monoref.markattr = luatexbase.new_attribute("monorefmark")
204
205 local HLIST = node.id("hlist")
206 local VLIST = node.id("vlist")
207 local GLYPH = node.id("glyph")
208 local GLUE = node.id("glue")
209
210 -----
211 -- Registration (all print/sprint straight back into the TeX stream)
212 -----
213 function monoref.known(name)
214   tex.print(monoref.labels[name] and 1 or 0)
215 end
216
217 function monoref.newslot(name, kind)
218   monoref.sid = monoref.sid + 1
219   monoref.slots[monoref.sid] = { name = name, font = font.current(), kind = kind }
220   tex.print(monoref.sid)
221 end
222
223 local function newmark(kind, key)
224   monoref.mid = monoref.mid + 1
225   monoref.marks[monoref.mid] = { kind = kind, key = key }
226   return monoref.mid
227 end
228
229 function monoref.label_begin(name, ref)
230   monoref.labels[name] = { ref = ref, page = 0 }
231   tex.print(newmark("label", name))
232 end
233
234 function monoref.ref_value(name)
235   local L = monoref.labels[name]
236   tex.sprint(L and L.ref or "??")
237 end

```

```

238
239 function monoref.toc_begin(level, num, title)
240   monoref.toc[#monoref.toc + 1] = { level = level, num = num, title = title, page = 0 }
241   tex.print(newmark("toc", #monoref.toc))
242 end
243
244 function monoref.toc_count()   tex.print(#monoref.toc)           end
245 function monoref.toc_level(i) tex.print(monoref.toc[i].level)   end
246 function monoref.toc_num(i)   tex.sprint(monoref.toc[i].num)     end
247 function monoref.toc_title(i) tex.sprint(monoref.toc[i].title)   end
248 function monoref.toc_page(i)  tex.print(monoref.toc[i].page)     end
249
250 -----
251 -- Marker scan: stamp every label / TOC entry with the TRUE page it is on
252 -----
253 local function scan(head, pg)
254   local n = head
255   while n do
256     local m = node.get_attribute(n, monoref.markattr)
257     if m and monoref.marks[m] then
258       local r = monoref.marks[m]
259       if r.kind == "label" and monoref.labels[r.key] then
260         monoref.labels[r.key].page = pg
261       elseif r.kind == "toc" and monoref.toc[r.key] then
262         monoref.toc[r.key].page = pg
263       end
264     end
265     local id = n.id
266     if id == HLIST or id == VLIST then scan(n.head, pg) end
267     n = n.next
268   end
269 end
270
271 function monoref.hold(reg, pg)
272   local b = tex.getbox(reg)
273   if not b then return end
274   local c = node.copy(b)
275   scan(c.head, pg)
276   local dest = (monoref.target == "toc") and monoref.toc_pages or monoref.held
277   dest[#dest + 1] = c
278 end
279
280 -----
281 -- Slot filling at flush (fixed width kept -> never reflows)
282 -----
283 local function glyphs_of(s, fid)
284   local fnt = font.getfont(fid)
285   local first, last
286   for i = 1, #s do
287     local code = s:byte(i)
288     local g = node.new(GLYPH)
289     g.font, g.char, g.subtype = fid, code, 0
290     if fnt and fnt.characters and fnt.characters[code] then
291       local ci = fnt.characters[code]

```

```

292     g.width, g.height, g.depth = ci.width or 0, ci.height or 0, ci.depth or 0
293   end
294   if first then last.next = g; g.prev = last; last = g
295   else first, last = g, g end
296 end
297 return first, last
298 end
299
300 local function value_of(rec)
301   if rec.kind == "total" then return tostring(monoref.total) end
302   local L = monoref.labels[rec.name]
303   if not L then return "??" end
304   if rec.kind == "page" then
305     return (L.page and L.page > 0) and tostring(L.page) or "??"
306   end
307   return L.ref
308 end
309
310 -- A replacement wider than its reserved slot cannot be absorbed by the
311 -- line's glue: warn, naming the slot and the template to widen.
312 local function warn_overflow(rec, val, over, where)
313   local what, tmpl
314   if rec.kind == "total" then
315     what, tmpl = "\\lastpage", "\\monorefpagetemplate"
316   elseif rec.kind == "page" then
317     what, tmpl = "\\pageref{" .. rec.name .. "}", "\\monorefpagetemplate"
318   elseif rec.name:sub(1, 5) == "mbib:" then
319     what, tmpl = "\\cite{" .. rec.name:sub(6) .. "}", "\\monorefcitetemplate"
320   else
321     what, tmpl = "\\ref{" .. rec.name .. "}", "\\monorefreftemplate"
322   end
323   texio.write_nl("term and log", string.format(
324     "Package monoref Warning: replacement '%s' for %s is %.1fpt wider",
325     val, what, over / 65536))
326   texio.write_nl("term and log", string.format(
327     "(monoref)                than its reserved slot on %s; the line", where))
328   texio.write_nl("term and log",
329     "(monoref)                may be overfull.  Widen " .. tmpl ..
330     " (or set \\monorefslotwidth).")
331   texio.write_nl("term and log", "")
332 end
333
334 -- Fill one placeholder hbox to the value's NATURAL width (the enclosing
335 -- line is re-justified afterwards so the reserved slack disappears).
336 local function fill_slot(hbox, id, where)
337   local val = value_of(monoref.slots[id])
338   local reserved = hbox.width
339   if hbox.head then node.flush_list(hbox.head); hbox.head = nil end
340   local gf = glyphs_of(val, monoref.slots[id].font)
341   local packed = node.hpack(gf or node.new(GLUE))
342   if packed.width > reserved + 1000 then -- ~0.015pt tolerance
343     warn_overflow(monoref.slots[id], val, packed.width - reserved, where)
344   end
345   hbox.head = packed.head

```

```

346 hbox.width      = packed.width
347 hbox.height     = packed.height
348 hbox.depth      = packed.depth
349 hbox.glue_set    = 0
350 hbox.glue_sign   = 0
351 hbox.glue_order  = 0
352 packed.head = nil
353 node.free(packed)
354 end
355
356 -- Re-pack a line to its original width so its interword glue absorbs the
357 -- space freed by shrinking a slot (keeps the line justified, no gap).
358 local function rejustify(hbox)
359   local packed = node.hpack(hbox.head, hbox.width, "exactly")
360   hbox.head     = packed.head
361   hbox.glue_set  = packed.glue_set
362   hbox.glue_sign = packed.glue_sign
363   hbox.glue_order = packed.glue_order
364   hbox.height   = packed.height
365   hbox.depth    = packed.depth
366   packed.head = nil
367   node.free(packed)
368 end
369
370 -- Patch every slot in a box; re-justify each line that contained one.
371 local function patch_box(box, where)
372   local direct = false
373   local n = box.head
374   while n do
375     local id = n.id
376     if id == HLIST or id == VLIST then
377       local a = node.get_attribute(n, monoref.slotattr)
378       if a and monoref.slots[a] then
379         fill_slot(n, a, where)
380         direct = true
381       else
382         patch_box(n, where)
383       end
384     end
385     n = n.next
386   end
387   if direct and box.id == HLIST then rejustify(box) end
388 end
389
390 -----
391 -- Finalisation and shipping
392 -----
393 function monoref.finalize()
394   monoref.total = #monoref.held
395   for i = 1, #monoref.toc_pages do
396     patch_box(monoref.toc_pages[i], "contents page " .. i)
397   end
398   for i = 1, #monoref.held do
399     patch_box(monoref.held[i], "body page " .. i)

```

```

400 end
401 monoref.queue = {}
402 for i = 1, #monoref.toc_pages do monoref.queue[#monoref.queue + 1] = monoref.toc_pages[i] e
403 for i = 1, #monoref.held do monoref.queue[#monoref.queue + 1] = monoref.held[i] e
404 end
405
406 function monoref.qcount() tex.print(#monoref.queue) end
407 function monoref.qpop() tex.setbox(monoref.shipreg, table.remove(monoref.queue, 1)) end
408
409 return monoref
410  $\langle$ /lua $\rangle$ 

```